

План-конспект к занятию №6

«Изучение новых слоёв сверточной нейросети»

Аннотация

№ занятия:	6
Тема занятия:	«Изучение новых слоёв сверточной нейросети»
Продолжительность занятия:	2 часа
Используемое оборудование:	Среда разработки NNWizard

Краткое описание содержания материала презентации:

Слайд 1. Титульный слайд, тема занятия.

Слайд 2-23. Слои свёрточной нейросети, с которыми мы еще не работали.

Слайд 24-26. Практическое задание.

Цель занятия: сформировать представление обучающихся о многообразии слоёв свёрточной нейросети.

Программа занятия:

1. Организационный этап.
2. Этап актуализации опорных знаний.
3. Этап формирования новых знаний и способов действия:
 - 3.1. Изучение новых слоёв свёрточной нейросети по разделам.
4. Этап применения знаний и формирование умений: практическая работа.
5. Подведение итогов. Рефлексия.

Методы, подходы и формы обучения: на достижение целей занятия направлены используемые педагогом методы. Используются следующие методы учебно-

познавательной деятельности: словесный, методы и приёмы самостоятельной работы. Используется групповая форма работы.

Выбранные методы позволяют реализовать цели системно-деятельностного подхода. Форма проведения занятий проводится с учетом возрастных особенностей и способствует формированию личностных, познавательных и регулятивных универсальных учебных действий.

Ход занятия

Организационный момент.

Этап формирования новых знаний и способов действия.

Слайд 1. Титульный слайд, тема занятия.

Тема занятия: “ Изучение новых слоёв сверточной нейросети”.

Слайд 2- 23. Слои свёрточной нейросети, с которыми мы еще не работали.

Вы могли заметить, что в программе NNWizard есть еще очень много слоев, которые мы не рассматривали. Давайте кратко разберем их. Начнем с **раздела “Активация”**.

Раздел “Активация”

В **разделе “Активация”** есть ряд слоев, которые мы не использовали в работе. Среди них: **“LeakyReLU”, “PReLU”, “ELU”, “ThresholdedReLU”**.

1. Leaky ReLU (рис. 1):

Leaky ReLU является вариацией стандартной функции ReLU.

ReLU очень проста и часто используется в нейронных сетях. Она просто возвращает 0 для всех отрицательных входных значений и само значение для всех неотрицательных входных значений. Таким образом, график ReLU имеет

"ступеньку" в точке 0, и наклон графика для всех отрицательных значений равен нулю.

Leaky ReLU сохраняет положительные значения без изменений, а для отрицательных значений возвращает небольшой отрицательный наклон, что может помочь избежать проблемы "мертвых нейронов" и улучшить обучение модели.



Рисунок 1 – Слой Leaky ReLU в NNWizard

2. PReLU (рис. 2):

PReLU - это обобщение Leaky ReLU, где отрицательный наклон является обучаемым параметром, в отличие от фиксированного значения, как в Leaky ReLU.

PReLU позволяет модели учить оптимальные значения отрицательных наклонов для каждого нейрона.



Рисунок 2 – Слой PReLU в NNWizard

3. ELU (рис.3):

ELU также является вариацией ReLU. Она имеет отрицательный наклон для отрицательных входов и экспоненциальную функцию для положительных входов.

ELU может помочь в борьбе с проблемой "умерших" нейронов и обладает свойствами нормализации. Нормализация в данном случае означает сохранение баланса в значениях, т.е. препятствует стремлению к нулю.



Рисунок 3 – Слой ELU в NNWizard

4. **Thresholded ReLU** (рис.4):

Thresholded ReLU обнуляет все значения, которые меньше определенного порога, и передает положительные значения без изменений.

Thresholded ReLU может быть полезной в определенных сценариях, где важно подавлять маленькие значения.

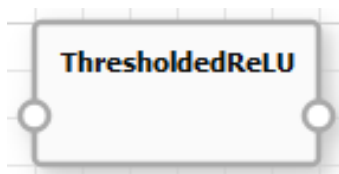


Рисунок 4 – Слой Thresholded ReLU в NNWizard

Выбор функции активации зависит от конкретной задачи и данных, поэтому часто рекомендуется экспериментировать с различными функциями, чтобы определить, какая наилучшим образом соответствует вашей модели.

Раздел “Свёртка”

В разделе “Свёртка” есть ряд слоев, которые мы не использовали в работе.

Среди них: **“Conv2DTranspose”, “DepthwiseConv2D”, “SeparableConv2d”**.

Давайте разберем подробнее:

1. **Conv2DTranspose** (рис. 5):

Этот слой выполняет операцию, обратную операции свертки (Conv2D). Он используется для увеличения размерности карты признаков.

Conv2DTranspose обычно используется в архитектурах генеративных моделей, таких как GANs (Generative Adversarial Networks) для увеличения разрешения изображений.

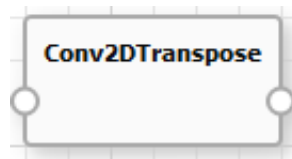


Рисунок 5 – Слой Conv2DTranspose в NNWizard

2. DepthwiseConv2D (рис. 6):

Этот слой выполняет свертку по каждому каналу входных данных отдельно. Он не смешивает каналы, что делает его вычислительно более эффективным по сравнению с обычным Conv2D.

DepthwiseConv2D может быть полезным в случаях, когда необходимо сохранить различные характеристики в разных каналах.

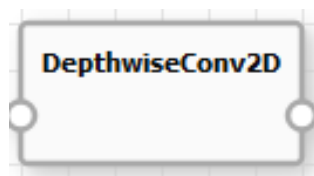


Рисунок 6 – Слой DepthwiseConv2D в NNWizard

3. SeparableConv2D (рис. 7):

Этот слой является комбинацией DepthwiseConv2D и обычного сверточного слоя. Он применяет depthwise свертку, а затем 1x1 свертку для объединения выходов.

SeparableConv2D также направлен на снижение вычислительной сложности, сохраняя при этом способность модели извлекать значимые признаки.

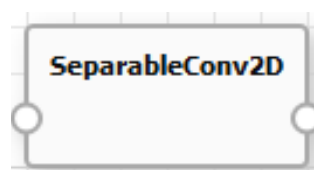


Рисунок 7 – Слой SeparableConv2D в NNWizard

Каждый из этих слоев имеет свои применения в зависимости от задачи и требований модели. Например, Conv2DTranspose может использоваться для генерации изображений, а DepthwiseConv2D и SeparableConv2D - для эффективного извлечения признаков.

Раздел “Нормализация”

В разделе “Нормализация” есть ряд слоев, которые мы не использовали в работе. Среди них: “**BatchNormalization**”, “**LayerNormalization**”.

1. **Batch Normalization** (рис. 8):

BatchNormalization является методом нормализации, применяемым к активациям нейронов внутри каждого мини-батча обучающих данных.

Мини-батч — это подмножество батча, которое используется для вычислений на каждом шаге обучения.

Он целенаправленно нормализует входные данные, чтобы среднее значение стало близким к нулю, а стандартное отклонение - близким к единице.

BatchNormalization улучшает стабильность и ускоряет обучение нейронных сетей, позволяя использовать более высокие скорости обучения.

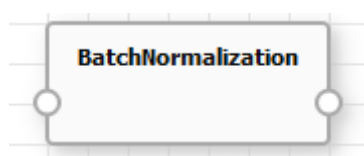


Рисунок 8 – Слой Batch Normalization в NNWizard

2. **Layer Normalization** (рис. 9):

LayerNormalization нормализует активации внутри каждого нейрона, рассматривая их как отдельные слои. В отличие от BatchNormalization, LayerNormalization не зависит от размера мини-батча и применяется независимо для каждого примера данных.

LayerNormalization полезна, когда размеры мини-батчей варьируются или когда работают с данными переменной длины.

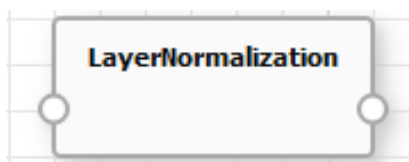


Рисунок 9 – Слой Layer Normalization в NNWizard

Оба этих слоя нормализации являются методами регуляризации и могут способствовать стабильности и эффективности обучения глубоких нейронных сетей. Выбор между ними обычно зависит от контекста задачи и характеристик данных. BatchNormalization чаще используется в сверточных и полносвязных слоях, тогда как LayerNormalization может быть полезной в случаях, где размеры мини-батчей изменчивы или когда идёт работа с вложенными последовательностями данных.

Раздел “Пулинг”

В разделе “Пулинг” есть ряд слоев, которые мы не использовали в работе. Среди них: “AveragePooling2D”, “GlobalAveragePooling2D”.

Разберем подробнее:

1. AveragePooling2D (рис. 10):

AveragePooling2D выполняет операцию усреднения по области пулинга. Для каждой области пулинга он вычисляет среднее значение активаций в этой области и использует его в качестве выходного значения. Этот слой помогает уменьшить размерность входных данных, усредняя информацию в каждой области пулинга.

AveragePooling2D обычно используется для уменьшения размера карт признаков, снижения вычислительной сложности и улучшения инвариантности к масштабу и переводу.

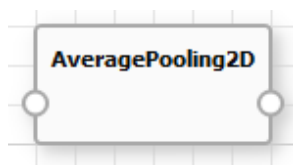


Рисунок 10 – Слой AveragePooling2D в NNWizard

2. **GlobalAveragePooling2D** (рис. 11):

GlobalAveragePooling2D выполняет аналогичную операцию усреднения, но вместо того, чтобы применять усреднение к каждой области пулинга, оно выполняет усреднение по всему пространству признаков. Для каждого канала входных данных вычисляется среднее значение по всем пикселям этого канала.

GlobalAveragePooling2D часто используется в архитектурах глубоких нейронных сетей перед полносвязными слоями или слоями классификации, чтобы уменьшить размерность признаков до одного значения на канал.

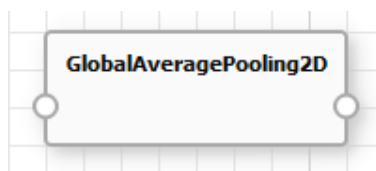


Рисунок 11 – Слой GlobalAveragePooling2D в NNWizard

Оба эти слоя позволяют уменьшить размерность данных, улучшая вычислительную эффективность и уменьшая количество параметров в модели. Выбор между ними зависит от архитектуры сети и требований к обработке признаков.

Раздел “Регуляризация”

В разделе “Регуляризация” есть ряд слоев, которые мы не использовали в работе. Среди них: “ActivityRegularization”, “ AlphaDropout ”, “Dropout”,

"GaussianDropout", "GaussianNoise", "SpatialDropout2D". Давайте кратко о них поговорим.

1. **ActivityRegularization** (рис. 12):

ActivityRegularization просто добавляет штраф к общей активации нейронов в слое. Это помогает предотвратить переобучение, убеждая сеть оставаться более разреженной (большинство элементов в слое имеют нулевые значения) в активации.

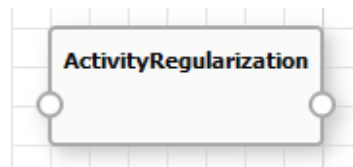


Рисунок 12 – Слой ActivityRegularization в NNWizard

2. **AlphaDropout** (рис. 13):

AlphaDropout - это вариант слоя Dropout, где случайные нейроны выключаются с учетом альфа-распределения (вероятность выключения для каждого нейрона будет более гибкой, чем в обычном Dropout). Это может быть более эффективным в некоторых сценариях, чем обычный Dropout.

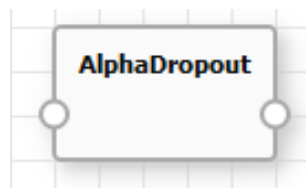


Рисунок 13 – Слой AlphaDropout в NNWizard

3. **Dropout** (рис. 14):

Dropout случайным образом "выключает" (устанавливает в ноль) определенное количество нейронов в слое во время обучения. Человек данный процесс проконтролировать не может. Это помогает предотвратить переобучение и улучшает обобщение модели на новые данные.



Рисунок 14 – Слой Dropout в NNWizard

4. **GaussianDropout** (рис. 15):

GaussianDropout - это вариант Dropout, где значения "выключаемых" нейронов заменяются случайными значениями из гауссовского распределения (т.е. значения не обязательно становятся нулевыми, могут принимать различные значения). Это должно помочь в сохранении более непрерывных характеристик слоя.

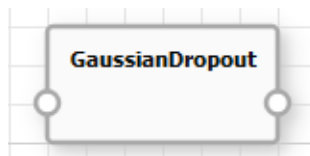


Рисунок 15 – Слой GaussianDropout в NNWizard

5. **GaussianNoise** (рис. 16):

Вместо того чтобы обучаться исключительно на "чистых" данных, модель обучается на данных, к которым добавлен случайный шум (в данном контексте шум – это некоторые случайные, непредсказуемые изменения или добавки, которые могут появиться в данных. В контексте изображений это может быть небольшое колебание яркости пикселей).

Этот шум может помочь модели лучше справляться с различными вариантами входных данных и делать её более устойчивой к потенциальным шумам или изменениям в реальных данных.

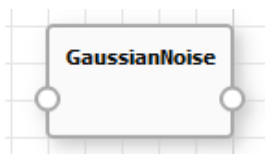


Рисунок 16 – Слой GaussianNoise в NNWizard

6. **SpatialDropout2D** (рис. 17):

SpatialDropout2D - это вариант Dropout для двумерных данных (например, изображений). В отличие от обычного Dropout, он выключает целые каналы признаков, что сохраняет пространственные корреляции в данных.

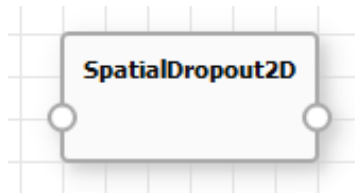


Рисунок 17 – Слой SpatialDropout2D в NNWizard

Эти слои регуляризации предназначены для борьбы с переобучением и повышения обобщающей способности моделей машинного обучения. Выбор конкретного слоя зависит от задачи, данных и архитектуры модели. Введение некоторой степени регуляризации может быть полезным при обучении глубоких нейронных сетей.

Раздел “Изменение формы”

В разделе “Изменение формы” есть ряд слоев, которые мы не использовали в работе. Среди них: “**Cropping2D**”, “**RepeatVector**”, “**Reshape**”, “**UpSampling2D**”, “**ZeroPadding2D**”.

Разберем подробнее:

1. **Cropping2D** (рис. 18):

Cropping2D позволяет обрезать входные данные вдоль каждого измерения. Это может быть полезно, например, когда нужно убрать нежелательные края изображения.



Рисунок 18 – Слой Cropping2D в NNWizard

2. RepeatVector (рис. 19):

RepeatVector повторяет входные данные заданное количество раз по заданному измерению. Это полезно, когда необходимо повторить последовательность для согласования размерности (формы) с другими данными.

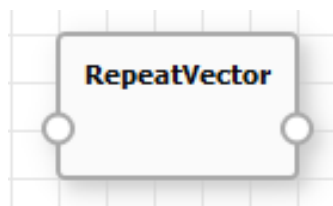


Рисунок 19 – Слой RepeatVector в NNWizard

3. Reshape (рис. 20):

Reshape изменяет форму (размерность) входных данных. Это может быть полезно, например, при подготовке данных для полносвязных слоев.

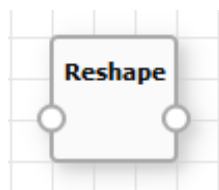


Рисунок 20 – Слой Reshape в NNWizard

4. UpSampling2D (рис. 21):

UpSampling2D — это операция, которая увеличивает размер входных данных, используя метод интерполяции. Давайте разберем, что это значит:

Увеличение размера означает увеличение размеров изображения или данных. Например, если у нас есть изображение 4x4 пикселя, то после увеличения размера оно может стать 8x8 пикселей.

Интерполяция - это метод, который используется для упрощения оценки значений на основе существующих. Когда мы увеличиваем размер данных, нам нужно заполнить новые пиксели. Интерполяция помогает определить значения этих пикселей.

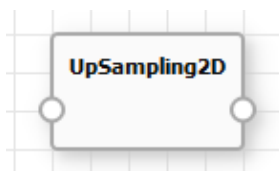


Рисунок 21 – Слой Reshape в NNWizard

5. ZeroPadding2D (рис. 22):

ZeroPadding2D добавляет нули по краям изображения. Это может быть полезно при обработке сверточных слоев, чтобы управлять размером выходных данных.

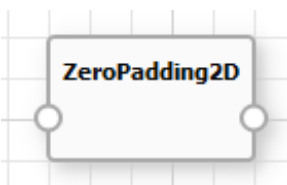


Рисунок 22 – Слой ZeroPadding2D в NNWizard

Эти слои изменения формы могут быть полезны в различных сценариях, таких как подготовка данных, обработка изображений и создание архитектур нейронных сетей. Выбор конкретного слоя зависит от задачи и требований вашей модели.

Этап применения знаний и формирование умений: практическая работа.

Задание 1. Измените структуру уже созданной вами нейросети или откройте проект из папки «project». Чтобы улучшить работу нейросети, используйте изученные на сегодняшнем занятии слои. Обучите нейросеть заново.

В качестве примера проекта вы можете использовать файл «Презентация для обучающихся».

Задание 2. Заполните таблицу. В левом столбце - названия слоев нейросети, вам нужно описать назначение и преимущества каждого из них.

Закрепление изученного учебного материала и рефлексия

1. Какие результаты вы получили после обучения вашей нейронной сети?
2. Достаточно ли хорошо работает обученная вами нейросеть? Почему?
3. Улучшила ли нейросеть свои показания после того, как мы изменили ее структуру? Почему?

Использованная и рекомендованная литература:

1. Сверточный слой: методы оптимизации, основанные на матричном умножении. [Электронный ресурс]. URL: <https://habr.com/ru/articles/448436/> (дата обращения: 11.12.2023)
2. Все, что вы хотели знать о слое пулинга в нейронных сетях [Электронный ресурс]. URL: <https://nauchniestati.ru/spravka/sloj-pulinga/> (дата обращения: 11.12.2023)
3. Сверточная нейронная сеть, часть 1: структура, топология, функции активации и обучающее множество . [Электронный ресурс]. URL: <https://habr.com/ru/articles/348000/>